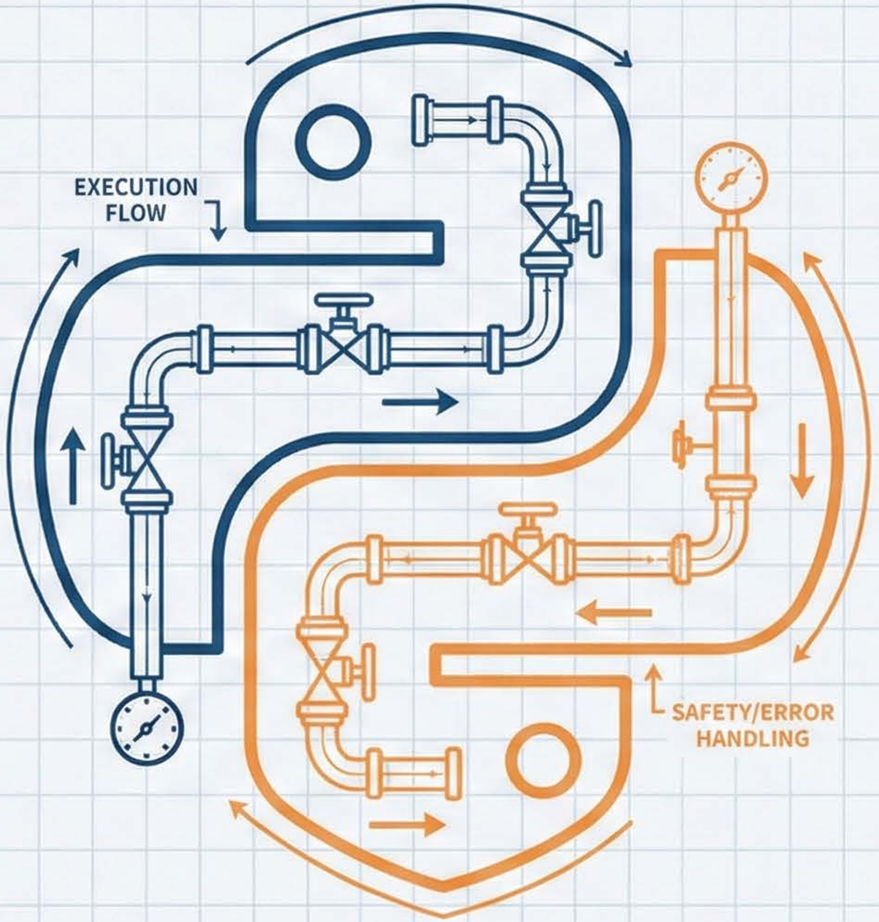
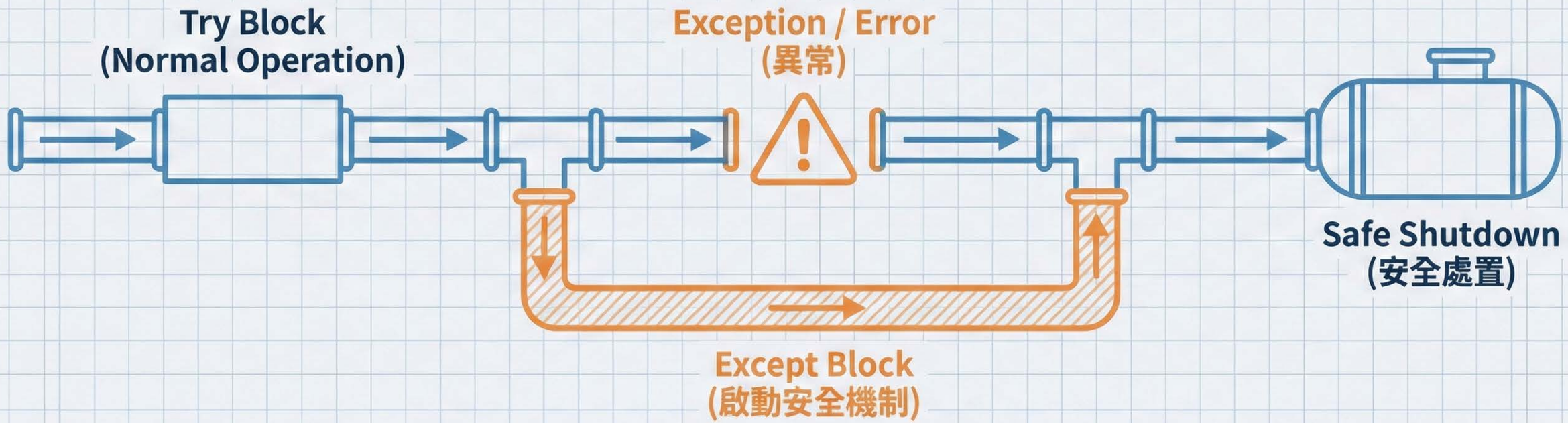
例外處理 (Exception Handling)：數位化工工廠安全連鎖系統

如何運用 Try-Except 架構設計穩健的化工程式 與防止系統崩潰



NotebookLM

程式執行的「旁路系統」



Try Block

Try Block：嘗試執行高風險操作（如反應器進料）。

Exception (Error)

Exception (Error)：發生異常（如管線阻塞、除以零）。

Except Block

Except Block：捕獲錯誤並執行替代方案，確保穩健性 (Robustness)。

NotebookLM

基礎語法：設計您的第一道防護網 (Basic Syntax)

Risky Operation (高風險)

```
flow_rate = 0
residence_time = 100 / flow_rate
print('計算完成')
```

ZeroDivisionError!
程式直接崩潰
(Crash)

Interlock Active (防護啟動)

```
try:
    flow_rate = 0
    residence_time = 100 / flow_rate
except ZeroDivisionError:
    print('錯誤：流量不能為 0')
    residence_time = -1
```

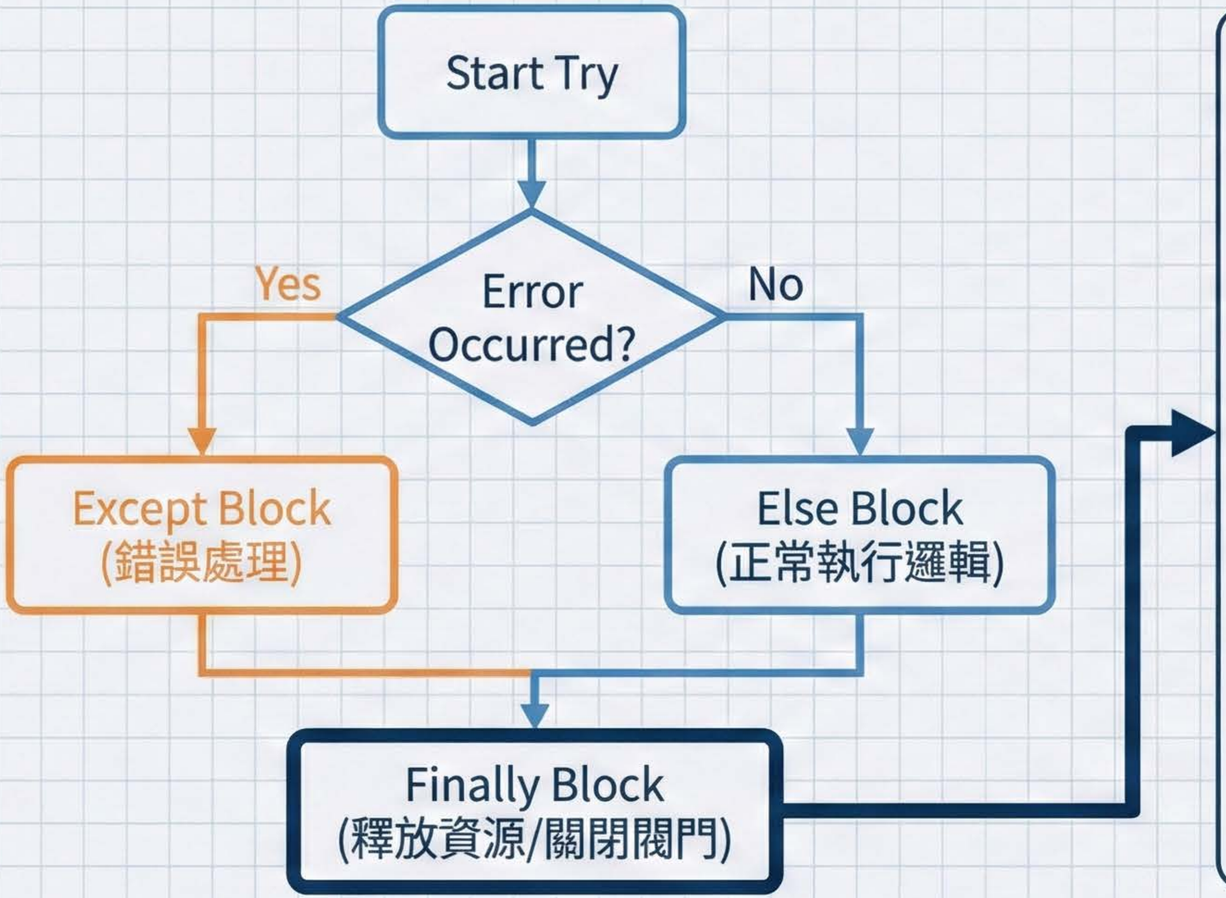
捕獲錯誤，
執行替代方案，
程式繼續運行

Try 區塊包含可能出錯的代碼；Except 區塊定義錯誤發生時的應對措施。

NotebookLM

完整控制流程：Else 與 Finally

無論反應是否成功，最後都必須關閉閥門 (Cleanup Phase)

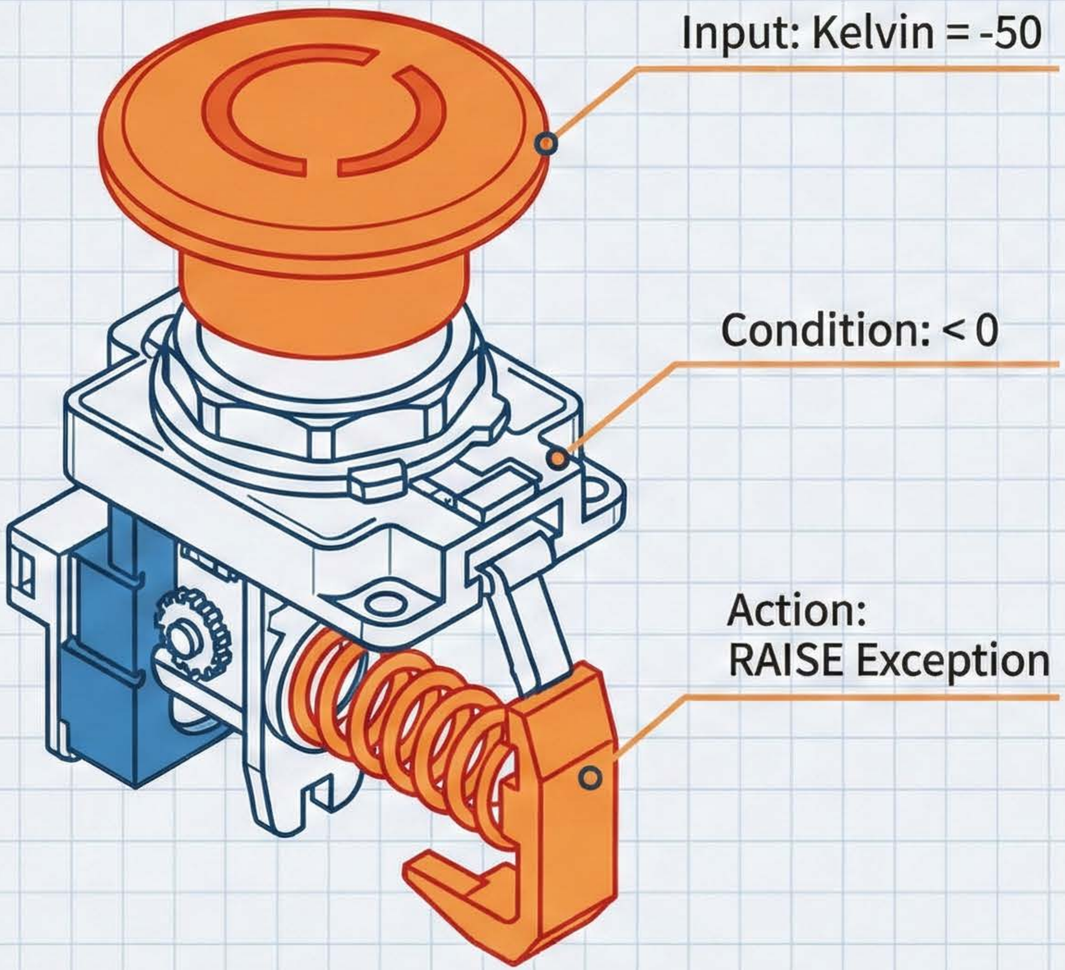



```python
try:
    file = open('reactor_log.txt', 'r')
    data = file.read()
except FileNotFoundError:
    print('找不到日誌檔案')
else:
    print('讀取成功，開始分析')
finally:
    # 關鍵：無論成功失敗，務必執行
    if 'file' in locals():
        file.close()
        print('檔案連結已關閉')
```

NotebookLM

主動防禦：使用 Raise 拋出例外 (Active Defense)

不要等待系統崩潰，偵測到危險輸入立即手動觸發警報



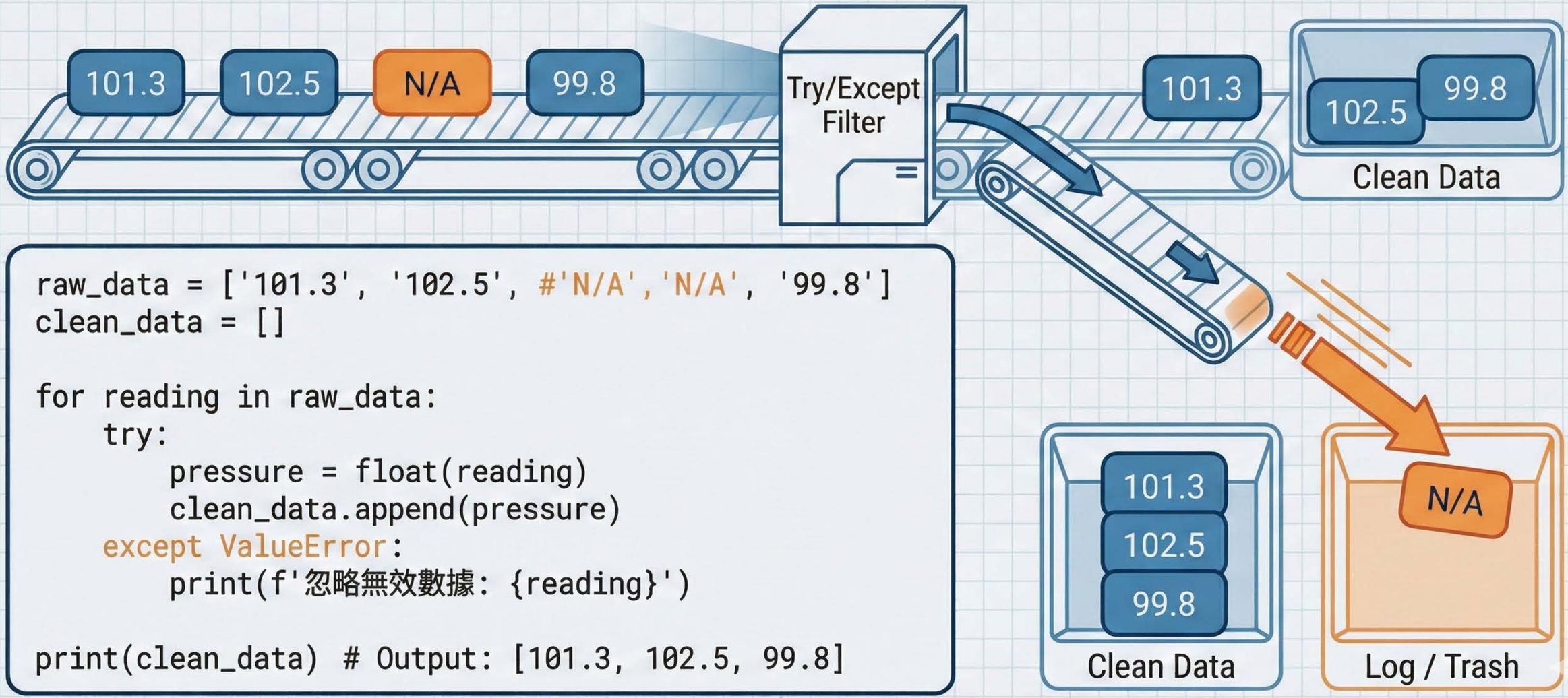

```
def check_temperature(kelvin):
    if kelvin < 0:
        # 物理上不可能的溫度
        raise ValueError('錯誤：絕對溫度不能為負值！')
    return kelvin + 273.15

try:
    check_temperature(-50)
except ValueError as e:
    print(f'警報：{e}')
```

Raise 用於強制執行商業運輯或物理限制 (Enforcing Physical Constraints)。

NotebookLM

化工實務應用：數據驗證 (Data Validation)

過濾感測器串流中的無效數據 (Filtering Bad Data)




```python
raw_data = ['101.3', '102.5', #'N/A','N/A', '99.8']
clean_data = []

for reading in raw_data:
    try:
        pressure = float(reading)
        clean_data.append(pressure)
    except ValueError:
        print(f'忽略無效數據: {reading}')

print(clean_data) # Output: [101.3, 102.5, 99.8]
```

NotebookLM

穩健性設計檢查表 (Robustness Design Checklist)

“Python 的例外處理機制，就是您程式碼中的工業安全閥。”